Vault Guardians Audit Report

Version 1.0

Cyfrin.io

April 6, 2026

Vault Guardians Audit Report

fervidflame

April 6, 2026

Prepared by: fervidflame

Lead Auditors: fervidflame

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
    * **[H-1] Lack of UniswapV2 slippage protection in `UniswapAdapter::_uniswapInvest` enables frontrunners to steal profits**
    * **[H-2] `ERC4626::totalAssets` checks the balance of vault's underlying asset even when the asset is invested, resulting in incorrect values being returned**
    * **[H-3] Guardians can infinitely mint `VaultGuardianToken`s and take over DAO, stealing DAO fees and maliciously setting parameters**

    * **[H-4] The stake price, `VaultGuardiansBase::GUARDIAN_FEE`, is not collected in the function `VaultGuardiansBase::becomeGuardian` causing anyone to become a guardian.**
    * **[H-5] Doubling the amount of `amountADesired` can cause `addLiquidity` to revert.**
    * **[H-6] There is no burning function in `VaultGuardianToken` contract. Therefore, a former guardian will retain their voting rights in the DAO.**
  - Medium
    * **[M-1] Potentially incorrect voting period and delay in governor may affect governance.**
  - Low
    * **[L-1] Incorrect vault name and symbol.**
    * **[L-2] Unassigned return value when divesting AAVE funds.**
    * **[L-3] Missing Events in Critical Functions**
    * **[L-4] Incorrect custom error name in the function `VaultGuardianBase::quitGuaridian`.**
    * **[L-5] Events Emitted After State Changes.**
  - Informational
    * **[I-1] The `IInvestableUniverseAdapter` interface is never been used in the protocol.**
    * **[I-2] The variable `VaultGuardiansBase::GUARDIAN_FEE` is never been used in the protocol.**
    * **[I-3] These custom errors were never used in the protocol.**

Protocol Summary

This protocol allows users to deposit certain ERC20s into an ERC4626 vault managed by a human being, or a `vaultGuardian`. The goal of a `vaultGuardian` is to manage the vault in a way that maximizes the value of the vault for the users who have despoited money into the vault.

Disclaimer

The fervidflame team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

Table 1: We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The findings described in this document correspond the following commit hash:

```
XXX
```

Scope

- In Scope:

```
./src/
#-- abstract
|   #-- AStaticTokenData.sol
|   #-- AStaticUSDCData.sol
|   #-- AStaticWethData.sol
#-- dao
|   #-- VaultGuardianGovernor.sol
|   #-- VaultGuardianToken.sol
#-- interfaces
|   #-- IVaultData.sol
|   #-- IVaultGuardians.sol
|   #-- IVaultShares.sol
|   #-- InvestableUniverseAdapter.sol
#-- protocol
|   #-- VaultGuardians.sol
|   #-- VaultGuardiansBase.sol
|   #-- VaultShares.sol
|   #-- investableUniverseAdapters
```

```
|        #-- AaveAdapter.sol
|        #-- UniswapAdapter.sol
#-- vendor
    #-- DataTypes.sol
    #-- IPool.sol
    #-- IUniswapV2Factory.sol
    #-- IUniswapV2Router01.sol
```

- Solc Version: 0.8.20
- Chain(s) to deploy contract to: Ethereum
- Tokens:
  - weth: https://etherscan.io/token/0xc02aaa39b223fe8d0a0e5c4f27ead9083c756cc2
  - link: https://etherscan.io/token/0x514910771af9ca656af840dff83e8264ecf986ca
  - usdc: https://etherscan.io/token/0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48

Roles

There are 4 main roles associated with the system.

- *Vault Guardian DAO*: The org that takes a cut of all profits, controlled by the `VaultGuardianToken`. The DAO that controls a few variables of the protocol, including:
  - `s_guardianStakePrice`
  - `s_guardianAndDaoCut`
  - And takes a cut of the ERC20s made from the protocol
- *DAO Participants*: Holders of the `VaultGuardianToken` who vote and take profits on the protocol
- *Vault Guardians*: Strategists/hedge fund managers who have the ability to move assets in and out of the investable universe. They take a cut of revenue from the protocol.
- *Investors*: The users of the protocol. They deposit assets to gain yield from the investments of the Vault Guardians.

Executive Summary

The Vault Guardians project takes novel approaches to work ERC-4626 into a hedge fund of sorts, but makes some large mistakes on tracking balances and profits.

Issues found

Table 2: List and quantity of severities.

Severity	Number of issues found
High	6
Medium	1
Low	5
Info	3
Gas	0
Total	15

Findings

High

[H-1] Lack of UniswapV2 slippage protection in `UniswapAdapter::_uniswapInvest` enables frontrunners to steal profits

Description: In `UniswapAdapter::_uniswapInvest` the protocol swaps half of an ERC20 token so that they can invest in both sides of a Uniswap pool. It calls the `swapExactTokensForTokens` function of the `UnisapV2Router01` contract , which has two input parameters to note:

```
    function swapExactTokensForTokens(
        uint256 amountIn,
@>      uint256 amountOutMin,
        address[] calldata path,
        address to,
@>      uint256 deadline
    )
```

The parameter `amountOutMin` represents how much of the minimum number of tokens it expects to return. The `deadline` parameter represents when the transaction should expire.

As seen below, the `UniswapAdapter::_uniswapInvest` function sets those parameters to `0` and `block.timestamp`:

```
    uint256[] memory amounts = i_uniswapRouter.swapExactTokensForTokens(
        amountOfTokenToSwap,
@>      0,
        s_pathArray,
        address(this),
@>      block.timestamp
    );
```

Impact: This results in either of the following happening: - Anyone (e.g., a frontrunning bot) sees this transaction in the mempool, pulls a flashloan and swaps on Uniswap to tank the price before the swap happens, resulting in the protocol executing the swap at an unfavorable rate. - Due to the lack of a deadline, the node who gets this transaction could hold the transaction until they are able to profit from the guaranteed swap.

Proof of Concept:

1. User calls `VaultShares::deposit` with a vault that has a Uniswap allocation.
    1. This calls `_uniswapInvest` for a user to invest into Uniswap, and calls the router's `swapExactTokensForTokens` function.
2. In the mempool, a malicious user could:
    1. Hold onto this transaction which makes the Uniswap swap
    2. Take a flashloan out
    3. Make a major swap on Uniswap, greatly changing the price of the assets
    4. Execute the transaction that was being held, giving the protocol as little funds back as possible due to the `amountOutMin` value set to 0.

This could potentially allow malicious MEV users and frontrunners to drain balances.

Recommended Mitigation:

For the deadline issue, we recommend the following:

DeFi is a large landscape. For protocols that have sensitive investing parameters, add a custom parameter to the `deposit` function so the Vault Guardians protocol can account for the customizations of DeFi projects that it integrates with.

In the `deposit` function, consider allowing for custom data.

```
- function deposit(uint256 assets, address receiver) public override(ERC4626, IERC4626) isActive returns (uint256) {
+ function deposit(uint256 assets, address receiver, bytes customData) public override(ERC4626, IERC4626) isActive returns (uint256) {
```

This way, you could add a `deadline` to the Uniswap swap, and also allow for more DeFi custom integrations.

For the `amountOutMin` issue, we recommend one of the following:

1. Do a price check on something like a Chainlink price feed before making the swap, reverting if the rate is too unfavorable.
2. Only deposit 1 side of a Uniswap pool for liquidity. Don't make the swap at all. If a pool doesn't exist or has too low liquidity for a pair of ERC20s, don't allow investment in that pool.

Note that these recommendation require significant changes to the codebase.

[H-2] ERC4626::totalAssets checks the balance of vault's underlying asset even when the asset is invested, resulting in incorrect values being returned

Description: The `ERC4626::totalAssets` function checks the balance of the underlying asset for the vault using the `balanceOf` function.

```
function totalAssets() public view virtual returns (uint256) {
    return _asset.balanceOf(address(this));
}
```

However, the assets are invested in the investable universe (Aave and Uniswap) which means this will never return the correct value of assets in the vault.

Impact: This breaks many functions of the `ERC4626` contract: - `totalAssets` - `convertToShares` - `convertToAssets` - `previewWithdraw` - `withdraw` - `deposit`

All calculations that depend on the number of assets in the protocol would be flawed, severely disrupting the protocol functionality.

Proof of Concept:

Code

Add the following code to the `VaultSharesTest.t.sol` file.

```
function testWrongBalance() public {
    // Mint 100 ETH
    weth.mint(mintAmount, guardian);
    vm.startPrank(guardian);
    weth.approve(address(vaultGuardians), mintAmount);
    address wethVault = vaultGuardians.becomeGuardian(allocationData);
    wethVaultShares = VaultShares(wethVault);
    vm.stopPrank();

    // prints 3.75 ETH
```

```
    console.log(wethVaultShares.totalAssets());

    // Mint another 100 ETH
    weth.mint(mintAmount, user);
    vm.startPrank(user);
    weth.approve(address(wethVaultShares), mintAmount);
    wethVaultShares.deposit(mintAmount, user);
    vm.stopPrank();

    // prints 41.25 ETH
    console.log(wethVaultShares.totalAssets());
}
```

Recommended Mitigation: Do not use the OpenZeppelin implementation of the `ERC4626` contract. Instead, natively keep track of users total amounts sent to each protocol. Potentially have an automation tool or some incentivised mechanism to keep track of protocol's profits and losses, and take snapshots of the investable universe.

This would take a considerable re-write of the protocol.

[H-3] Guardians can infinitely mint `VaultGuardianTokens` and take over DAO, stealing DAO fees and maliciously setting parameters

Description: Becoming a guardian comes with the perk of getting minted Vault Guardian Tokens (vgTokens). Whenever a guardian successfully calls `VaultGuardiansBase::becomeGuardian` or `VaultGuardiansBase::becomeTokenGuardian`, `_becomeTokenGuardian` is executed, which mints the caller `i_vgToken`.

```
    function _becomeTokenGuardian(IERC20 token, VaultShares tokenVault)
        private returns (address) {
        s_guardians[msg.sender][token] = IVaultShares(address(
            tokenVault));
@>      i_vgToken.mint(msg.sender, s_guardianStakePrice);
        emit GuardianAdded(msg.sender, token);
        token.safeTransferFrom(msg.sender, address(this),
            s_guardianStakePrice);
        token.approve(address(tokenVault), s_guardianStakePrice);
        tokenVault.deposit(s_guardianStakePrice, msg.sender);
        return address(tokenVault);
    }
```

Guardians are also free to quit their role at any time, calling the `VaultGuardianBase::quitGuardian` function. The combination of minting vgTokens, and freely being able to quit, results in users being able to farm vgTokens at any time.

Impact: Assuming the token has no monetary value, the malicious guardian could accumulate

tokens until they can overtake the DAO. Then, they could execute any of these functions of the `VaultGuardians` contract:

```
  "sweepErc20s(address)": "942d0ff9",
  "transferOwnership(address)": "f2fde38b",
  "updateGuardianAndDaoCut(uint256)": "9e8f72a4",
  "updateGuardianStakePrice(uint256)": "d16fe105",
```

Proof of Concept:

1. User becomes WETH guardian and is minted vgTokens.
2. User quits, is given back original WETH allocation.
3. User becomes WETH guardian with the same initial allocation.
4. Repeat to keep minting vgTokens indefinitely.

Code

Place the following code into `VaultGuardiansBaseTest.t.sol`

```
    function testDaoTakeover() public hasGuardian hasTokenGuardian {
        address maliciousGuardian = makeAddr("maliciousGuardian");
        uint256 startingVoterUsdcBalance = usdc.balanceOf(maliciousGuardian);
        uint256 startingVoterWethBalance = weth.balanceOf(maliciousGuardian);
        assertEq(startingVoterUsdcBalance, 0);
        assertEq(startingVoterWethBalance, 0);

        VaultGuardianGovernor governor = VaultGuardianGovernor(payable(vaultGuardians.owner()));
        VaultGuardianToken vgToken = VaultGuardianToken(address(governor.token()));

        // Flash loan the tokens, or just buy a bunch for 1 block
        weth.mint(mintAmount, maliciousGuardian); // The same amount as the other guardians
        uint256 startingMaliciousVGTokenBalance = vgToken.balanceOf(maliciousGuardian);
        uint256 startingRegularVGTokenBalance = vgToken.balanceOf(guardian);
        console.log("Malicious vgToken Balance:\t", startingMaliciousVGTokenBalance);
        console.log("Regular vgToken Balance:\t", startingRegularVGTokenBalance);

        // Malicious Guardian farms tokens
        vm.startPrank(maliciousGuardian);
        weth.approve(address(vaultGuardians), type(uint256).max);
        for (uint256 i; i < 10; i++) {
```

```
            address maliciousWethSharesVault = vaultGuardians.becomeGuardian(allocationData);
            IERC20(maliciousWethSharesVault).approve(
                address(vaultGuardians),
                IERC20(maliciousWethSharesVault).balanceOf(maliciousGuardian)
            );
            vaultGuardians.quitGuardian();
        }
        vm.stopPrank();

        uint256 endingMaliciousVGTokenBalance = vgToken.balanceOf(maliciousGuardian);
        uint256 endingRegularVGTokenBalance = vgToken.balanceOf(guardian);
        console.log("Malicious vgToken Balance:\t", endingMaliciousVGTokenBalance);
        console.log("Regular vgToken Balance:\t", endingRegularVGTokenBalance);
    }
```

Recommended Mitigation: There are a few options to fix this issue:

1. Mint vgTokens on a vesting schedule after a user becomes a guardian.
2. Burn vgTokens when a guardian quits.
3. Simply don't allocate vgTokens to guardians. Instead, mint the total supply on contract deployment.

[H-4] The stake price, `VaultGuardiansBase::GUARDIAN_FEE`, is not collected in the function `VaultGuardiansBase::becomeGuardian` causing anyone to become a guardian.

Description: The stake price, `GUARDIAN_FEE`, is a constant variable that should be paid by any user wanting to become a vault guardian. The NATSPEC of the function `VaultGuardiansBase::becomeGuardian` states, "they [the user] have to send an ETH amount equal to the fee, and a WETH amount equal to the stake price". Yet, the function `becomeGuardian` does not collect the stake price and in addition, it is not `payable` and does not check `msg.value`.

```
uint256 private constant GUARDIAN_FEE = 0.1 ether;
.
.
.

@> function becomeGuardian(AllocationData memory wethAllocationData) external returns (address) {
```

Impact: The protocol loses its core functionality of collecting fees per guardian and creating a DAO

based on paid membership, thus consequently allowing malicious actors to create an infinite amount of vaults for free.

Proof of Concept (Proof of Code):

Proof of Code

Add the following code to the `VaultGuardiansBaseTest.t.sol` file.

```
function testBecomeGuardianWithoutPayingFee() public {
    // Give user some WETH to attempt to become a guardian without paying the fee
    deal(address(weth), user, stakePrice);
    vm.startPrank(user);
    weth.approve(address(vaultGuardians), stakePrice);
    uint256 balanceBefore = address(vaultGuardians).balance;
    // User calls `becomeGuardian` function without sending the stake price in ETH
    vaultGuardians.becomeGuardian(allocationData);
    uint256 balanceAfter = address(vaultGuardians).balance;
    // Assert the stake price fee was NOT collected
    assertEq(balanceAfter, balanceBefore);
    vm.stopPrank();
}
```

Recommended Mitigation: Make `becomeGuardian` payable and require `msg.value` to be greater or equal to the `GUARDIAN_FEE`. Use the pre-exisiting custom error, `VaultGuardiansBase__FeeTooSmall(uint256 fee, uint256 requiredFee);`. This must be used because it is not used at all in the entire protocol.

```
- function becomeGuardian(AllocationData memory wethAllocationData) external returns (address) {
+ function becomeGuardian(AllocationData memory wethAllocationData) external payable returns (address) {
+       if (msg.value < GUARDIAN_FEE) {
            revert VaultGuardiansBase__FeeTooSmall(uint256 fee, uint256 requiredFee);
        }
        ...
        }
```

[H-5] Doubling the amount of `amountADesired` can cause `addLiquidity` to revert.

Description: In the function, `UniswapAdapter::_uniswapInvest`, the desired tokenA amount, `amountADesired`, is set to `amountOfTokenToSwap + amounts[0]`. However, `amounts[0]` represents the amount of input tokens already swapped. Essentially, this results in asking to deposit

roughly the full original token amount even though half was already swapped away. That can cause a revert of insufficient funds if the router tries to pull more than the contract holds.

```
function _uniswapInvest(IERC20 token, uint256 amount) internal {
    ...

 (uint256 tokenAmount, uint256 counterPartyTokenAmount, uint256 liquidity) = i_uniswapRouter.addLiquidity({
            tokenA: address(token),
            tokenB: address(counterPartyToken),
@>          amountADesired: amountOfTokenToSwap + amounts[0],
            amountBDesired: amounts[1],
            amountAMin: 0,
            amountBMin: 0,
            to: address(this),
            deadline: block.timestamp
        });
        ...
```

Impact: If the investment strategy is Hold: 0%, Uniswap: 1000%, Aave: 0%, it would result in a revert. For example, `addLiquidity` is called when a user wants to become a Vault Guardian.

Here is the sequence of events: `VaultGuardiansBase::becomeGuardian` calls -> `VaultGuardiansBase::_becomeTokenGuardian` calls -> `VaultShares::deposit` calls -> `VaultShares::_investFunds` calls -> `UniswapAdapter::_uniswapInvest` calls -> `IUniswapV2Router01::addLiquidity`

This can potentially cause a DoS for `VaultGuardiansBase::becomeGuardian` and `VaultShares::deposit`.

Proof of Concept (Proof of Code):

Proof of Code

Add the following code to the `VaultGuardiansBaseTest.t.sol` file.

```

AllocationData extremeUniswapAllocationData = AllocationData(0, 1000, 0);

function testUniswapAmountADesired() public {
        // Extreme allocation data for testing Uniswap amountADesired edge case
        // AllocationData extremeUniswapAllocationData = AllocationData(0, 1000, 0);

        // Give user some WETH to cover the stake price
        deal(address(weth), user, stakePrice);
        vm.startPrank(user);
        weth.approve(address(vaultGuardians), stakePrice);
```

```

        // Expect revert
        vm.expectRevert();
        vaultGuardians.becomeGuardian(extremeUniswapAllocationData);
        vm.stopPrank();
    }
```

Recommended Mitigation: Remove `amounts[0]`.

```
function _uniswapInvest(IERC20 token, uint256 amount) internal {
    ...

 (uint256 tokenAmount, uint256 counterPartyTokenAmount, uint256 liquidity) = i_uniswapRouter.addLiquidity({
            tokenA: address(token),
            tokenB: address(counterPartyToken),
-          amountADesired: amountOfTokenToSwap + amounts[0],
+          amountADesired: amountOfTokenToSwap,
            amountBDesired: amounts[1],
            amountAMin: 0,
            amountBMin: 0,
            to: address(this),
            deadline: block.timestamp
        });
        ...
```

[H-6] There is no burning function in `VaultGuardianToken` contract. Therefore, a former guardian will retain their voting rights in the DAO.

Description: There is no burning function in `VaultGuardianToken` contract and no burning of the vgTokens in the `VaultGuardiansBase::quitGuardian` function. Therefore, if a user does not want to be a guardian anymore, they will still retain their voting rights in the DAO.

```
contract VaultGuardianToken is ERC20, ERC20Permit, ERC20Votes, Ownable {
    constructor() ERC20("VaultGuardianToken", "VGT") ERC20Permit("VaultGuardianToken") Ownable(msg.sender) {}

    // The following functions are overrides required by Solidity.
    /**
     * @dev Move voting power when tokens are transferred.
     *
     * Emits a {IVotes-DelegateVotesChanged} event.
     */
    function _update(address from, address to, uint256 value) internal override(ERC20, ERC20Votes) {
        super._update(from, to, value);
    }
```

```

    function nonces(address ownerOfNonce) public view override(
        ERC20Permit, Nonces) returns (uint256) {
        return super.nonces(ownerOfNonce);
    }

    function mint(address to, uint256 amount) external onlyOwner {
        _mint(to, amount);
    }
}
```

Impact: This goes against the core functionality of the protocol - lack of paid membership in the DAO. Former guardians still retain their voting rights, despite leaving the DAO.

Recommended Mitigation: Add a burn function to the `VaultGuardianToken` contract and add that `burn` function to the `_quitGuardian` function to burn any vgTokens when somone quits being a guardian. Most likely, a mapping of vgTokens per guardian would have to be created. Use that mapping to keep track of the minting and burning of vgTokens.

Medium

[M-1] Potentially incorrect voting period and delay in governor may affect governance.

The `VaultGuardianGovernor` contract, based on OpenZeppelin Contract's Governor, implements two functions to define the voting delay (`votingDelay`) and period (`votingPeriod`). The contract intends to define a voting delay of 1 day, and a voting period of 7 days. It does it by returning the value `1 days` from `votingDelay` and `7 days` from `votingPeriod`. In Solidity these values are translated to number of seconds.

However, the `votingPeriod` and `votingDelay` functions, by default, are expected to return number of blocks. Not the number seconds. This means that the voting period and delay will be far off what the developers intended, which could potentially affect the intended governance mechanics.

Consider updating the functions as follows:

```
function votingDelay() public pure override returns (uint256) {
-   return 1 days;
+   return 7200; // 1 day
}

function votingPeriod() public pure override returns (uint256) {
-   return 7 days;
+   return 50400; // 1 week
}
```

Low

[L-1] Incorrect vault name and symbol.

When new vaults are deployed in the `VaultGuardianBase::becomeTokenGuardian` function, symbol and vault name are set incorrectly when the `token` is equal to `i_tokenTwo`. Consider modifying the function as follows, to avoid errors in off-chain clients reading these values to identify vaults.

```
else if (address(token) == address(i_tokenTwo)) {
    tokenVault =
    new VaultShares(IVaultShares.ConstructorData({
        asset: token,
-       vaultName: TOKEN_ONE_VAULT_NAME,
+       vaultName: TOKEN_TWO_VAULT_NAME,
-       vaultSymbol: TOKEN_ONE_VAULT_SYMBOL,
+       vaultSymbol: TOKEN_TWO_VAULT_SYMBOL,
        guardian: msg.sender,
        allocationData: allocationData,
        aavePool: i_aavePool,
        uniswapRouter: i_uniswapV2Router,
        guardianAndDaoCut: s_guardianAndDaoCut,
        vaultGuardian: address(this),
        weth: address(i_weth),
        usdc: address(i_tokenOne)
    }));
```

Also, add a new test in the `VaultGuardiansBaseTest.t.sol` file to avoid reintroducing this error, similar to what's done in the test `testBecomeTokenGuardianTokenOneName`.

[L-2] Unassigned return value when divesting AAVE funds.

The `AaveAdapter::_aaveDivest` function is intended to return the amount of assets returned by AAVE after calling its `withdraw` function. However, the code never assigns a value to the named return variable `amountOfAssetReturned`. As a result, it will always return zero.

While this return value is not being used anywhere in the code, it may cause problems in future changes. Therefore, update the `_aaveDivest` function as follows:

```
function _aaveDivest(IERC20 token, uint256 amount) internal returns (
    uint256 amountOfAssetReturned) {
-       i_aavePool.withdraw({
+       amountOfAssetReturned = i_aavePool.withdraw({
            asset: address(token),
            amount: amount,
            to: address(this)
```

```
        });
}
```

[L-3] Missing Events in Critical Functions

Description: Important functions lack event emissions: `AaveAdapter::_aaveInvest` and `AaveAdapter::_aaveDivest`

Recommended Mitigation: Add appropriate events for off-chain tracking; make sure the events are added before external calls.

[L-4] Incorrect custom error name in the function `VaultGuardianBase::quitGuaridian`.

Description: If a vault guardian wants to quit being a guardian, they call the function `VaultGuardianBase::quitGuaridian` which will revert if the guardian has non-WETH vaults. It reverts with the custom error, `VaultGuardiansBase__CantQuitWethWithThisFunction()`, which is misleading.

Recommended Mitigation: Delete that line and use the correct customer error that is already written in the protocol.

```
 function quitGuardian() external onlyGuardian(i_weth) returns (uint256) {
        if (_guardianHasNonWethVaults(msg.sender)) {
-           revert VaultGuardiansBase__CantQuitWethWithThisFunction();
+           revert VaultGuardiansBase__CantQuitGuardianWithNonWethVaults(address guardianAddress);
        }
        return _quitGuardian(i_weth);
    }
```

[L-5] Events Emitted After State Changes.

Description: The following functions emit events after state changes: - `VaultShares.sol::updateHoldingAllocation` - `VaultGuardians.sol::updateGuardianStakePrice` - `VaultGuardians.sol::updateGuardianAndDaoCut`-`UniswapAdapter::_uniswapInvest` - `UniswapAdapter::_uniswapDivest`

For best CEI practices, place event emissions before state changes.

Recommended Mitigation: Consider standardizing event emissions before state changes.

Informational

[I-1] The `IInvestableUniverseAdapter` interface is never been used in the protocol.

Recommended Mitigation: Delete the interface.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.20;

import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

-interface IInvestableUniverseAdapter {
-    // function invest(IERC20 token, uint256 amount) external;
-    // function divest(IERC20 token, uint256 amount) external;
}
```

[I-2] The variable `VaultGuardiansBase::GUARDIAN_FEE` is never been used in the protocol.

Recommended Mitigation: Delee the variable or implement it in the protocol.

```
-   uint256 private constant GUARDIAN_FEE = 0.1 ether;
```

[I-3] These custom errors were never used in the protocol.

Consider using or removing the unused error.

4 Found Instances

- Found in src/protocol/VaultGuardians.sol Line: 43

```
    error VaultGuardians__TransferFailed();
```

- Found in src/protocol/VaultGuardiansBase.sol Line: 46

```
    error VaultGuardiansBase__NotEnoughWeth(uint256 amount,
        uint256 amountNeeded);
```

- Found in src/protocol/VaultGuardiansBase.sol Line: 48

```
    error VaultGuardiansBase__CantQuitGuardianWithNonWethVaults(
        address guardianAddress);
```

- Found in src/protocol/VaultGuardiansBase.sol Line: 51

```
    error VaultGuardiansBase__FeeTooSmall(uint256 fee, uint256
        requiredFee);
```